

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХМЕЛЬНИЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

Факультет інформаційних технологій

Кафедра комп'ютерної інженерії та інформаційних систем

Лабораторна робота №1

з дисципліни

«Теорія, проектування та моделювання спеціалізованих комп'ютерних систем»

Виконав: студент 2 курсу, групи КІм-25-1

(Підпис)

Шуневич В.І.

Перевірив:

(Підпис)

Лисенко С.М.

Хмельницький – 2026

Лабораторна робота №1

Проектування спеціалізованої комп'ютерної системи відеоспостереження на базі Raspberry Pi з функцією виявлення руху

Мета: формування практичних навичок проектування та розробки спеціалізованих комп'ютерних систем, що здійснюють обробку відеоінформації в реальному часі із використанням одноплатної комп'ютерної системи Raspberry Pi та бібліотеки OpenCV.

Завдання:

1-й рівень

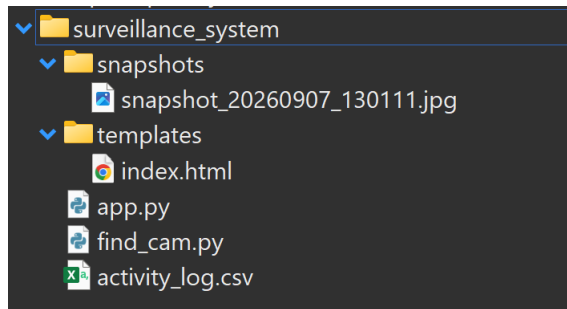
Виконати проектування та реалізувати спеціалізовану комп'ютерну систему відеоспостереження на базі Raspberry Pi з функцією виявлення руху відповідно до методичних рекомендацій.

2-й рівень

Виконати завдання першого рівня. Дослідити функціонування системи при різних конфігураціях. Додатково виконати завдання відповідно до варіанту:

1. Створити CSV-журнал активності, куди заносити час початку й завершення кожного руху.
2. Збереження моментів руху як знімки, тобто при кожному виявленні руху зберігати кадр з таймстемпом у директорію snapshots/.
3. Показувати в веб-інтерфейсі під відео, коли востаннє був зафіксований рух.
4. Зміна чутливості через веб-інтерфейс (наприклад у вигляді слайдери), що дозволить змінювати поріг `cv2.contourArea()` (тобто чутливість до руху).

Файлова структура:



Kod index.html:

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Відеоспостереження + Чутливість + Моніторинг</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      background: #181818;
      color: #ffffff;
      margin: 0;
      padding: 20px;
    }
    .container {
      max-width: 900px;
      margin: 0 auto;
    }
    .stream-box {
      display: inline-block;
      border: 3px solid #00e676;
      border-radius: 8px;
      overflow: hidden;
      background: #000;
      margin-bottom: 15px;
    }
    img {
      display: block;
      max-width: 100%;
      height: auto;
    }
    .panel {
      background: #242424;
      padding: 15px 20px;
      border-radius: 8px;
      border: 1px solid #383838;
      margin-bottom: 20px;
    }
```



```

    </p>
</div>

<!-- Графіки використання ресурсів -->
<div class="charts-grid">
    <div class="card">
        <h3>Завантаження CPU (%)</h3>
        <canvas id="cpuChart"></canvas>
    </div>
    <div class="card">
        <h3>Використання RAM (%)</h3>
        <canvas id="ramChart"></canvas>
    </div>
</div>
</div>

<script>
    // Відправка оновленого значення порогу на бекенд
    function updateSensitivity(val) {
        document.getElementById('sliderValue').innerText = val + " px";
        fetch('/set_sensitivity', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ threshold: parseInt(val) })
        });
    }

    // Конфігурація графіків Chart.js
    const MAX_POINTS = 15;

    const cpuCtx = document.getElementById('cpuChart').getContext('2d');
    const cpuChart = new Chart(cpuCtx, {
        type: 'line',
        data: {
            labels: [],
            datasets: [{
                label: 'CPU %',
                data: [],
                borderColor: '#ff5252',
                backgroundColor: 'rgba(255, 82, 82, 0.2)',
                fill: true,
                tension: 0.3
            }]
        },
        options: {
            scales: {
                y: { min: 0, max: 100, ticks: { color: '#aaa' } },
                x: { ticks: { color: '#aaa' } }
            },
            plugins: { legend: { display: false } }
        }
    });

```

```
const ramCtx = document.getElementById('ramChart').getContext('2d');
const ramChart = new Chart(ramCtx, {
  type: 'line',
  data: {
    labels: [],
    datasets: [{
      label: 'RAM %',
      data: [],
      borderColor: '#448aff',
      backgroundColor: 'rgba(68, 138, 255, 0.2)',
      fill: true,
      tension: 0.3
    }]
  },
  options: {
    scales: {
      y: { min: 0, max: 100, ticks: { color: '#aaa' } },
      x: { ticks: { color: '#aaa' } }
    },
    plugins: { legend: { display: false } }
  }
});
```

```
// Періодичне отримання метрик CPU/RAM
```

```
setInterval(async () => {
  try {
    const response = await fetch('/system_metrics');
    const data = await response.json();

    cpuChart.data.labels.push(data.time);
    cpuChart.data.datasets[0].data.push(data.cpu);

    ramChart.data.labels.push(data.time);
    ramChart.data.datasets[0].data.push(data.ram);

    if (cpuChart.data.labels.length > MAX_POINTS) {
      cpuChart.data.labels.shift();
      cpuChart.data.datasets[0].data.shift();
      ramChart.data.labels.shift();
      ramChart.data.datasets[0].data.shift();
    }

    cpuChart.update();
    ramChart.update();
  } catch (err) {
    console.error("Помилка отримання метрик:", err);
  }
}, 1000);
```

```
</script>
```

```
</body>
```

```
</html>
```

Kod app.py:

```
import os
import sys
import cv2
import time
import csv
import threading
import psutil
import numpy as np
import imutils
from datetime import datetime
from flask import Flask, render_template, Response, jsonify, request

app = Flask(__name__)

CSV_FILE = "activity_log.csv"

# Створення файлу журналу, якщо він відсутній
if not os.path.exists(CSV_FILE):
    with open(CSV_FILE, mode="w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["Start Time", "End Time", "Duration (s)"])

# Спільні ресурси та синхронізація
frame_to_show = np.zeros((480, 640, 3), dtype=np.uint8)
lock = threading.Lock()

# Змінна чутливості (поріг площі контуру)
min_contour_area = 10000

def get_camera_capture():
    backend = cv2.CAP_DSHOW if sys.platform.startswith("win") else cv2.CAP_ANY
    for idx in [0, 1]:
        cap = cv2.VideoCapture(idx, backend)
        if cap.isOpened():
            ret, test_frame = cap.read()
            if ret and test_frame is not None:
                print(f"☑ Камеру підключено під індексом: {idx}")
                return cap
            cap.release()
    print("✗ Не вдалося відкрити камеру. Перевірте дозволи в системі.")
    return None

def capture_and_detect_motion():
    global frame_to_show, min_contour_area

    video = get_camera_capture()
    if video is None:
        return

    time.sleep(1.0)
```

```

static_back = None
motion_start_time = None

while True:
    check, frame = video.read()
    if not check or frame is None:
        time.sleep(0.02)
        continue

    frame = imutils.resize(frame, width=640)
    current_time = datetime.now()
    timestamp_str = current_time.strftime("%Y-%m-%d %H:%M:%S")

    # Попередня обробка зображення
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (21, 21), 0)

    if static_back is None:
        static_back = gray.copy().astype("float")
        continue

    # Побудова фонової моделі та визначення різниці
    cv2.accumulateWeighted(gray, static_back, 0.05)
    diff_frame = cv2.absdiff(gray, cv2.convertScaleAbs(static_back))
    thresh_frame = cv2.threshold(diff_frame, 30, 255, cv2.THRESH_BINARY)[1]
    thresh_frame = cv2.dilate(thresh_frame, None, iterations=2)

    cnts, _ = cv2.findContours(thresh_frame.copy(), cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    # Отримання актуального значення чутливості під блокуванням
    with lock:
        threshold_area = min_contour_area

    motion = False
    for contour in cnts:
        if cv2.contourArea(contour) < threshold_area:
            continue
        motion = True
        (x, y, w, h) = cv2.boundingRect(contour)
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    # Фіксація подій у CSV-журнал
    if motion:
        if motion_start_time is None:
            motion_start_time = current_time
        else:
            if motion_start_time is not None:
                motion_end_time = current_time
                duration = round((motion_end_time - motion_start_time).total_seconds(),
2)

                with open(CSV_FILE, mode="a", newline="", encoding="utf-8") as f:
                    writer = csv.writer(f)

```



```

        writer.writerow([
            motion_start_time.strftime("%Y-%m-%d %H:%M:%S"),
            motion_end_time.strftime("%Y-%m-%d %H:%M:%S"),
            duration
        ])
        motion_start_time = None

    # Візуалізація інформації
    cv2.putText(frame, f"Motion: {'YES' if motion else 'NO'}", (10, 30),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255) if motion else (0, 255,
0), 2)
    cv2.putText(frame, timestamp_str, (10, 60),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
    cv2.putText(frame, f"Threshold: {threshold_area} px", (10, 90),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 1)

    with lock:
        frame_to_show = frame.copy()

    time.sleep(0.03)

video.release()
def generate():
    global frame_to_show
    while True:
        with lock:
            frame = frame_to_show.copy()
        ret, buffer = cv2.imencode('.jpg', frame)
        if not ret:
            time.sleep(0.01)
            continue
        yield (b'--frame\r\n'
              b'Content-Type: image/jpeg\r\n\r\n' + buffer.tobytes() + b'\r\n')
        time.sleep(0.03)
@app.route('/')
def index():
    return render_template('index.html')
@app.route('/video_feed')
def video_feed():
    return Response(generate(), mimetype='multipart/x-mixed-replace; boundary=frame')
# API: Оновлення порогу чутливості від повзунка
@app.route('/set_sensitivity', methods=['POST'])
def set_sensitivity():
    global min_contour_area
    data = request.get_json()
    if data and "threshold" in data:
        with lock:
            min_contour_area = int(data["threshold"])
        return jsonify({"status": "success", "threshold": min_contour_area})
    return jsonify({"status": "error"}), 400
# API: Відправка системних метрик CPU/RAM
@app.route('/system_metrics')
def system_metrics():

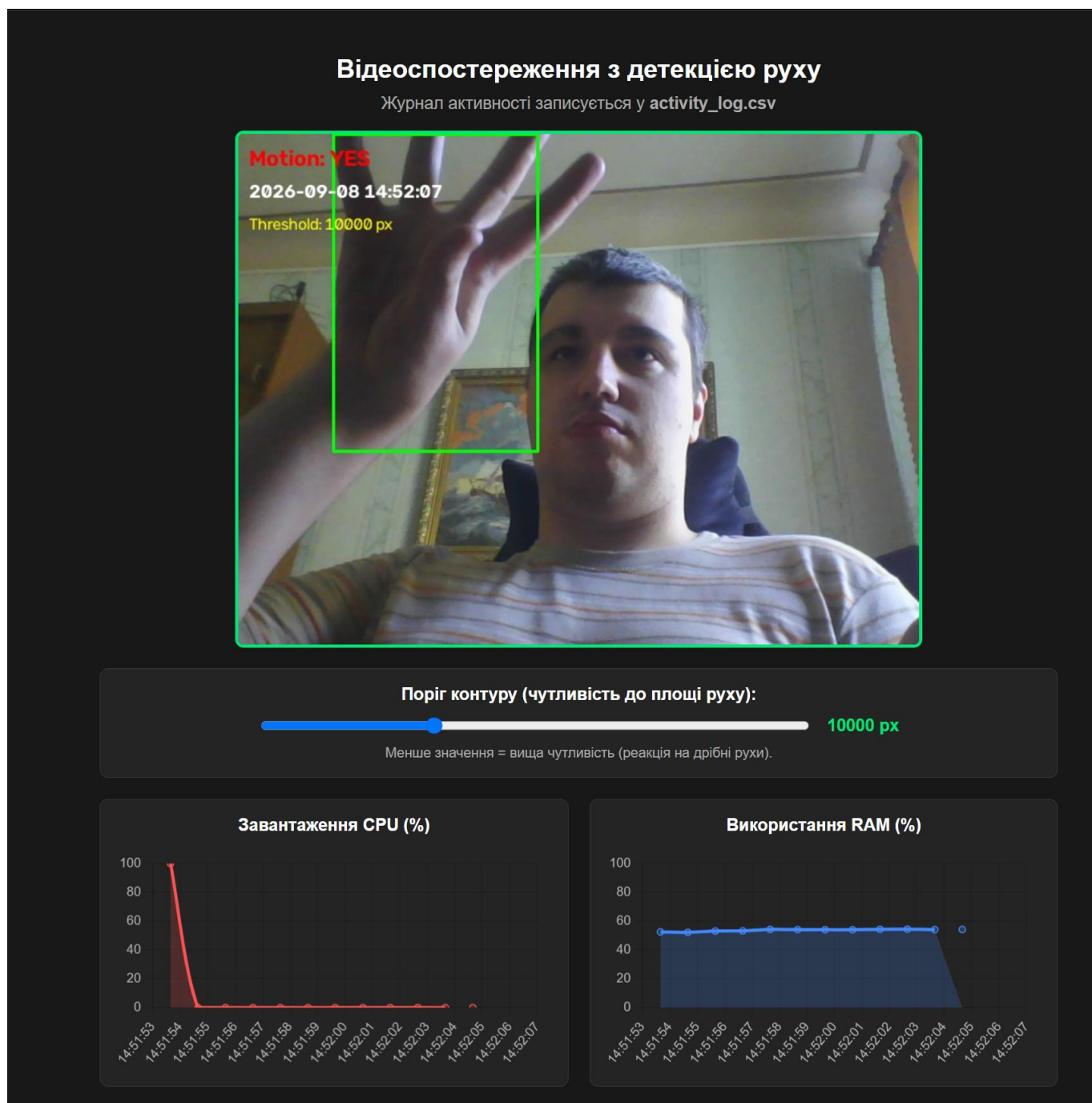
```

```

return jsonify({
    "time": datetime.now().strftime("%H:%M:%S"),
    "cpu": psutil.cpu_percent(interval=None),
    "ram": psutil.virtual_memory().percent
})
if __name__ == '__main__':
    psutil.cpu_percent(interval=None)
    threading.Thread(target=capture_and_detect_motion, daemon=True).start()
    app.run(host='0.0.0.0', port=5000, debug=False)

```

Скриншот виконання:



Висновок:

Під час виконання лабораторної роботи було спроектовано та реалізовано спеціалізовану комп'ютерну систему відеоспостереження в режимі реального часу з функцією виявлення руху. У ході роботи було закріплено практичні навички роботи з бібліотекою комп'ютерного зору OpenCV, а саме: застосування гаусового згладжування для усунення шуму, побудова динамічної адаптивної фонові моделі за допомогою функції `accumulateWeighted`, обчислення абсолютної різниці кадрів, порогова бінаризація, морфологічна обробка (дилатація) та пошук контурів рухомих об'єктів із фільтрацією за мінімальною площею. Завдяки веб-фреймворку Flask було забезпечено безперервну трансляцію обробленого потокового відео користувачеві через протокол Motion JPEG (multipart/x-mixed-replace). Багатопотокова організація програми дозволила розділити обчислювально складні алгоритми обробки відео та роботу мережевого інтерфейсу, що запобігає затримкам і зависанню веб-сервера.